

Weather Agent Debugging Documentation

1. Configuration Type Hints & Values

- **File & Component:** config.py (Config class)
- **Issue :** The application had a syntax error due to an invalid type assignment (TEMP_MIN = str = "Needs Debugging"). Additionally, it redeclared WEATHER_API_BASE_URL as an empty string at the bottom of the class, overriding the correct URL.
- **Resolution :** The syntax was corrected to standard Pydantic type hinting (TEMP_MIN: str = "Needs Debugging"). The WEATHER_API_BASE_URL was repopulated with the correct Open-Meteo endpoint ("https://api.open-meteo.com/v1/forecast").
- **Impact:** Resolves application crashes caused by invalid syntax and ensures the weather API can be successfully reached.

2. Temperature & Greeting Logic in Helper Functions

- **File & Component:** helper_functions.py (classify_temperature and get_greeting)
- **Issue :** The classify_temperature function contained a logic flaw (if temp_celsius: return config.TEMP_MIN) that would intercept any non-zero temperature and incorrectly return the debugging string instead of the actual classification. The get_greeting function relied on a strict integer check (if is_day == 1:).
- **Resolution :** The faulty if temp_celsius: block was completely removed, allowing the function to properly evaluate the temperature thresholds. The greeting function was updated to evaluate the boolean truthiness of the input (if bool(is_day):).
- **Impact:** The agent now accurately categorizes temperatures (e.g., "cold", "cool", "warm") and handles day/night boolean values more robustly in Python.

3. Location API Fallback System

- **File & Component:** nodes.py (fetch_location_data)
- **Issue :** The location node was fragile, relying entirely on a single provider (ipapi.co) without any fallback mechanisms if the service failed or rate-limited the user.
- **The Resolution :** Two new internal helper functions (_normalize_location_payload and _fetch_location_from_url) were added to standardize location data structures. The fetch_location_data node was refactored to iterate through a list of URLs, attempting ipwho.is if the primary provider fails.
- **Impact:** Significantly improves the agent's reliability and fault tolerance during the first step of the graph execution.

4. Formatting the Output String

- **File & Component:** nodes.py (generate_weather_info)

- **Issue** : The variable assignment for `wind_unit` was commented out, resulting in a literal, hardcoded string (`"wind_unit"`) being injected into the final output message.
- **Resolution** : The `wind_unit = units.get("windspeed", "km/h")` line was uncommented, and the f-string was fixed to actually evaluate the variable (`f"Wind: {windspeed} {wind_unit}"`). Bullet points were also changed from `•` to `-`, and the default temperature unit was changed from `"°C"` to `"C"`.
- **Impact**: The agent now dynamically displays the correct wind measurement metric to the user in a cleaner list format.

5. Graph Execution & Application Structure

- **File & Component**: Graph architecture vs. CLI runtime
- **Issue**: The graph building logic was housed in a file named `main.py`, but lacked an actual execution script to take user input or run the compiled graph.
- **Resolution**: The graph definition was logically separated into its own `graph.py` file. A new execution script (`main_2.py`) was introduced to prompt the user for their name, initialize the `WeatherAgentState`, invoke the graph, and print the resulting weather information to the console.
- **Impact**: Transitions the code from a static `LangGraph` definition into a fully interactive, executable command-line application.